

Spring AOP Pointcut Practice Questions

Application Structure

Use the following application structure while writing the pointcut expressions:

```
com. [redacted].student
├── controller
│   ├── StudentController
│   └── AdminController
├── service
│   ├── StudentService
│   ├── EmailService
│   └── impl
│       └── PaymentServiceImpl
├── repository
│   └── StudentRepository
└── model
    ├── Student
    └── PaymentRequest
```

Assume the following methods are present inside `StudentService`:

```
package com. [redacted].student.service;

import com. [redacted].student.model.Student;
```

```
import java.util.List;

public class StudentService {

    public Student getStudent(Long id) {
        return null;
    }

    public List<Student> getAllStudents() {
        return null;
    }

    public Student createStudent(Student student) {
        return null;
    }

    public Student updateStudent(Long id, Student student) {
        return null;
    }

    public void deleteStudent(Long id) {
    }

    public boolean studentExists(String email) {
        return false;
    }

    public void clearCache() {
    }
}
```

Section 1: Easy **execution()** Questions

Question 1: Match Every Method

Write an `execution()` pointcut that matches every interceptable of:

- Return type
 - Class
 - Method name
 - Number of parameters
 - Parameter types
-

Question 2: Match Every Method Named `getStudent`

Write an `execution()` pointcut that matches every method named:

```
getStudent
```

The method may:

- Belong to any class
 - Have any return type
 - Accept any number of parameters
-

Question 3: Match Methods Beginning with `get`

Write a pointcut that matches methods whose names begin with `get`.

It should match:

```
getStudent()  
getAllStudents()  
getStudentByEmail()  
getStudentCount()
```

It should not match:

```
createStudent()  
deleteStudent()  
studentExists()
```

Question 4: Match Methods Ending with

Write a pointcut that matches methods whose names end with `Student`.

It should match:

```
getStudent()  
createStudent()  
deleteStudent()  
updateStudent()
```

It should not match:

```
getStudentById()  
createStudentAccount()  
studentExists()
```

Question 5: Match Methods Containing `Student`

Write a pointcut that matches methods whose names contain the word `Student` anywhere in the method name.

It should match:

```
getStudent()  
getStudentById()  
createStudentAccount()  
updateStudentDetails()  
studentExists()
```

It should not match:

```
clearCache()  
sendEmail()  
processPayment()
```

Question 6: Match Methods with No Par

Write a pointcut that matches methods containing exactly zero parameters.

It should match:

```
getAllStudents()  
clearCache()  
generateReport()
```

It should not match:

```
getStudent(Long id)  
createStudent(Student student)
```

Question 7: Match Methods with Exactly One Parameter

Write a pointcut that matches methods containing exactly one parameter of any type.

It should match:

```
getStudent(Long id)  
createStudent(Student student)  
deleteStudent(Long id)  
studentExists(String email)
```

It should not match:

```
getAllStudents()  
clearCache()  
updateStudent(Long id, Student student)
```

Question 8: Match Methods with Exactly Parameter

Write a pointcut that matches methods accepting exactly one parameter of type:

```
java.lang.Long
```

It should match:

```
getStudent(Long id)  
deleteStudent(Long id)
```

It should not match:

```
createStudent(Student student)  
updateStudent(Long id, Student student)  
studentExists(String email)
```

Section 2: Intermediate `execution()` Questions

Question 9: Match Every Method in `StudentService`

Write an `execution()` pointcut that matches every method declared inside:

```
com. .... .student.service.StudentService
```

The methods may have:

- Any return type
- Any method name
- Any number of parameters

Question 10: Match Only `getStudent(Long`

Write a pointcut that matches only the following method:

```
public Student getStudent(Long id)
```

The expression should specify:

- The complete class name
- The exact method name
- Exactly one `Long` parameter
- Any return type

Class:

```
com._____.student.service.StudentService
```

Question 11: Match All Public Methods in `StudentService`

Write a pointcut that matches all public methods declared inside:

```
com._____.student.service.StudentService
```

The expression should not depend on:

- Method name
- Return type
- Number of parameters

Question 12: Match Methods Directly Ins Service Package

Write an `execution()` pointcut that matches methods declared directly inside:

```
com._____.student.service
```

It should match methods in:

```
com._____.student.service.StudentService  
com._____.student.service.EmailService
```

It should not match methods in:

```
com._____.student.service.impl.PaymentServiceImpl
```

Question 13: Match the Service Package and All Subpackages

Write an `execution()` pointcut that matches methods inside:

```
com._____.student.service
```

and all its subpackages.

It should match methods in:

```
com._____.student.service.StudentService  
com._____.student.service.EmailService  
com._____.student.service.impl.PaymentServiceImpl
```

Question 14: Match Public Service Meth with `get`

Write a pointcut that satisfies all the following conditions:

- The method must be public.
- The method name must begin with `get`.
- The method must belong to the service package or one of its subpackages.
- The method may accept any number of parameters.
- The method may return any type.

It should match:

```
StudentService.getStudent(Long id)
StudentService.getAllStudents()
PaymentServiceImpl.getPaymentStatus(Long id)
```

It should not match:

```
StudentController.getStudent(Long id)
StudentService.createStudent(Student student)
```

Question 15: Match Methods Whose First Parameter Is `Long`

Write a pointcut that matches methods whose first parameter is:

```
java.lang.Long
```

The method may have zero or more additional parameters after the first parameter.

It should match:

```
getStudent(Long id)

updateStudent(
    Long id,
    Student student
)

someMethod(
    Long id,
    String name,
    int age
)
```

It should not match:

```
createStudent(Student student)
studentExists(String email)
clearCache()
```

Question 16: Match Methods Returning **void**

Write a pointcut that matches methods whose return type is:

```
void
```

The class, method name and parameters may be anything.

It should match:

```
deleteStudent(Long id)
clearCache()
sendEmail(String message)
processPayment(PaymentRequest request)
```

It should not match:

```
getStudent(Long id)
studentExists(String email)
```

Section 3: `within()` and Annotation Pointcuts

Question 17: Match the Service Layer Using `within()`

Write a pointcut using only `within()` that matches every method declared inside:

```
com._____.student.service
```

and all its subpackages.

Do not use `execution()`.

Question 18: Combine `within()` and `execution()`

Write a pointcut that matches public methods beginning with `get`, but only inside the service layer.

Combine:

```
within()
```

and:

```
execution()
```

using:

```
&&
```

It should match:

```
StudentService.getStudent(Long id)
StudentService.getAllStudents()
PaymentServiceImpl.getPaymentStatus(Long id)
```

It should not match:

```
StudentController.getStudent(Long id)
StudentService.createStudent(Student student)
```

Question 19: Match Classes Annotated with **@Audited**

Assume the following annotation exists:

```
com._____.student.annotation.Audited
```

Example:

```
@Audited
@Service
public class PaymentServiceImpl {

    public void processPayment() {

    }

    public void refundPayment() {

    }

}
```

Write a pointcut using **@within()** that matches methods declared inside classes annotated with **@Audited**.

Question 20: Match Methods Annotated with **@TrackExecutionTime**

Assume the following annotation exists:

```
com. .... .student.annotation.TrackExecutionTime
```

Example:

```
@TrackExecutionTime
public Student createStudent(Student student) {
    return student;
}
```

Write a pointcut that matches methods directly annotated with `@TrackExecutionTime`.
Use the appropriate annotation-based pointcut designator.

Additional Tricky Practice Questions

Question A: Exclude One Method

Write a pointcut that matches all methods inside the service layer except methods named:

```
clearCache()
```

Use:

- `within()`
- `execution()`
- `&&`
- `!`

It should match:

```
getStudent(Long id)
createStudent(Student student)
```

```
deleteStudent(Long id)
```

It should not match:

```
clearCache()
```

Question B: Match Controllers or Services

Write a pointcut that matches methods belonging to either:

```
com. .... .student.controller
```

or:

```
com. .... .student.service
```

Include all subpackages.

Use:

```
||
```

It should match:

```
controller.StudentController  
controller.AdminController  
service.StudentService  
service.EmailService  
service.impl.PaymentServiceImpl
```

It should not match:

```
repository.StudentRepository
```

Question C: Match Public Methods Except Repositories

Write a pointcut that matches public methods inside the com

```
com. .... .student
```

However, exclude all methods inside:

```
com. .... .student.repository
```

and its subpackages.

Use:

- `within()`
- `execution()`
- `&&`
- `!`

Question D: Match a Bean by Name

Write a pointcut that matches every method belonging to the Spring bean named:

```
studentService
```

Use:

```
bean( )
```

Question E: Match Beans Ending with `Service`

Write a pointcut that matches Spring beans whose names end with:

```
Service
```

It should match bean names such as:

```
studentService  
emailService  
paymentService  
notificationService
```

Use a wildcard with `bean()`.

Question F: Match a Runtime `Student` Argument

Write a pointcut that matches methods receiving a `Student` object at runtime.

Use:

```
args()
```

with:

```
com. .... .student.model.Student
```

Example method:

```
public Student createStudent(Student student) {  
    return student;  
}
```

Question G: Match an Argument Annotated with `@SensitiveData`

Assume the following annotation exists:

```
com. .... .student.annotation.SensitiveData
```

Assume the runtime argument class is annotated:

```
@SensitiveData
public class PaymentRequest {
}
```

Write a pointcut that matches methods when the runtime class of an argument is annotated with `@SensitiveData`.

Use:

```
@args()
```

Question H: Match the Actual Target Bean Type

Write a pointcut that matches methods when the actual Spring bean behind the proxy is:

```
com. .... .student.service.StudentService
```

Use:

```
target()
```

Question I: Match the Spring Proxy Interface

Assume `StudentService` implements the following interface:

```
com. .... .student.service.StudentOperations
```

```
public interface StudentOperations {  
  
    Student getStudent(Long id);  
  
    Student createStudent(Student student);  
}
```

Write a pointcut that checks whether the Spring proxy implements `StudentOperations`.

Use:

```
this()
```

Question J: Tricky Parameter Matching

Write a separate `execution()` pointcut for each requirement.

J1. Exactly Zero Parameters

Match methods containing no parameters.

It should match:

```
getAllStudents()  
clearCache()
```

J2. Exactly One Parameter of Any Type

It should match:

```
getStudent(Long id)  
createStudent(Student student)  
studentExists(String email)
```

J3. Zero or More Parameters

It should match all the following:

```
clearCache()  
  
getStudent(Long id)  
  
updateStudent(  
    Long id,  
    Student student  
)
```

J4. Exactly Two Parameters of Any Type

It should match:

```
updateStudent(Long id, Student student)  
searchStudent(String name, Integer age)
```

It should not match:

```
clearCache()  
getStudent(Long id)
```

J5. First Parameter Is **Long**

The method may have zero or more parameters after the first **Long** parameter.

It should match:

```
getStudent(Long id)  
  
updateStudent(  
    Long id,
```

```
        Student student
    )

    someMethod(
        Long id,
        String name,
        int age
    )
```

J6. Last Parameter Is **Student**

The method may have zero or more parameters before the final **Student** parameter.

It should match:

```
createStudent(Student student)

updateStudent(
    Long id,
    Student student
)

saveStudent(
    String createdBy,
    Long departmentId,
    Student student
)
```

It should not match:

```
getStudent(Long id)
studentExists(String email)
```

Challenge Questions

Challenge 1: Public Service Methods Except Operations

Write a pointcut that matches:

- Public methods
- Inside the service layer
- Except methods whose names begin with `delete`

It should match:

```
getStudent(Long id)
createStudent(Student student)
updateStudent(Long id, Student student)
```

It should not match:

```
deleteStudent(Long id)
deleteAllStudents()
```

Challenge 2: Getter Methods with No Parameters

Write a pointcut that matches methods satisfying both conditions:

- The method name begins with `get`.
- The method contains no parameters.

It should match:

```
getAllStudents()
getStudentCount()
getApplicationName()
```

It should not match:

```
getStudent(Long id)
createStudent(Student student)
```

```
// Write the pointcut expression here
```

Challenge 3: Methods Returning **Student**

Write a pointcut that matches methods returning:

The method may:

- Belong to any class
- Have any name
- Accept any number of parameters

It should match:

```
getStudent(Long id)
createStudent(Student student)
updateStudent(Long id, Student student)
```

It should not match:

```
getAllStudents()
deleteStudent(Long id)
studentExists(String email)
```

Challenge 4: Methods with **Long** and **Student** Parameters

Write a pointcut that matches methods containing exactly these two parameters in this order:

```
Long  
Student
```

It should match:

```
updateStudent(Long id, Student student)
```

It should not match:

```
createStudent(Student student)  
getStudent(Long id)  
someMethod(Student student, Long id)
```

Challenge 5: Audited Public Service Methods

Write a pointcut that matches methods satisfying all the following conditions:

- The method is public.
- The method belongs to the service layer.
- The declaring class is annotated with `@Audited`.

Use:

- `within()`
- `execution()`
- `@within()`
- `&&`

Annotation:

```
com._____.student.annotation.Audited
```

Spring AOP Pointcut Practice Questions Answer Key

Some questions may have more than one valid solution. The answers are clear and commonly used answers based on the given

Section 1: Easy `execution()` Answers

Answer 1: Match Every Method

```
execution(* *(..))
```

* → any return type
* → any method name
(..) → zero or more parameters

Answer 2: Match Every Method Named `getStudent`

```
execution(* getStudent(..))
```

Answer 3: Match Methods Beginning with `get`

```
execution(* get*(..))
```

The `get*` pattern matches any method name beginning with `get`.

Answer 4: Match Methods Ending with `Student`

```
execution(* *Student(..))
```

The `*Student` pattern matches method names ending with `Stu`

Answer 5: Match Methods Containing `S`

```
execution(* *Student*(..))
```

The `*Student*` pattern matches method names containing `Student` anywhere in the name.

Answer 6: Match Methods with No Parameters

```
execution(* *())
```

Empty parentheses mean that the method must contain exactly zero parameters.

Answer 7: Match Methods with Exactly One Parameter

```
execution(* *(*))
```

`(*)` → exactly one parameter of any type

Answer 8: Match Methods with Exactly One `Long` Parameter

```
execution(* *(java.lang.Long))
```

This matches methods containing exactly one parameter, and that parameter must be `Long`.

Section 2: Intermediate **executi**

Answers

Answer 9: Match Every Method in **Stude**

```
execution(* com. .student.service.Stud  
(..))
```

* → any return type
StudentService → exact declaring class
* → any method name
(..) → any number of parameters

Answer 10: Match Only **getStudent(Long)**

```
execution(  
    * com. .student.service.StudentService.getStudent  
(java.lang.Long)  
)
```

One-line form:

```
execution(* com. .student.service.StudentService.getS  
tudent(java.lang.Long))
```

Answer 11: Match All Public Methods in **StudentService**

```
execution(  
    public * com. .student.service.StudentService.*
```

```
(..)  
)
```

One-line form:

```
execution(public * com. .student.service.*(..))
```

Answer 12: Match Methods Directly Inside the Service Package

```
execution(* com. .student.service.*.*(..))
```

The first `*` after `service` matches classes directly inside the package.

It does not include subpackages such as:

```
com. .student.service.impl
```

Answer 13: Match the Service Package and All Subpackages

```
execution(* com. .student.service.*.*(..))
```

The `..` after `service` allows zero or more package levels.

Therefore, it includes:

```
service.StudentService  
service.EmailService  
service.impl.PaymentServiceImpl
```

Answer 14: Match Public Service Methods with **get**

```
execution(  
    public * com. .student.service.*.  
)
```

One-line form:

```
execution(public * com. .student.service.*.get*(..))
```

public	→ public methods only
*	→ any return type
service.*	→ service package and its subpackages
get*	→ method name beginning with get
(..)	→ any number of parameters

Answer 15: Match Methods Whose First Parameter Is **Long**

```
execution(* *(java.lang.Long, ..))
```

java.lang.Long	→ the first parameter must be Long
..	→ zero or more additional parameters

This matches both:

```
getStudent(Long id)
```

and:

```
updateStudent(Long id, Student student)
```

Answer 16: Match Methods Returning **v**

```
execution(void *(..))
```

void → exact return type
* → any method name
(..) → any number of parameters

Section 3: **within()** and Annotation Answers

Answer 17: Match the Service Layer Using **within()**

```
within(com. student.service..*)
```

This matches methods declared inside the service package and all its subpackages.

Answer 18: Combine **within()** and **execution()**

```
within(com. student.service..*)  
&& execution(public * get*(..))
```

One-line form:

```
within(com. student.service..*) && execution(public  
* get*(..))
```

Both conditions must be satisfied:

1. The method must be inside the service layer.
2. The method must be public and begin with **get**.

Answer 19: Match Classes Annotated wi

```
@within(com. .student.annotation.Audit
```

This matches methods declared inside classes annotated wit

Answer 20: Match Methods Annotated with `@TrackExecutionTime`

```
@annotation(  
    com. .student.annotation.TrackExecutionTime  
)
```

One-line form:

```
@annotation(com. .student.annotation.TrackExecutionTi  
me)
```

`@annotation()` checks the annotation placed directly on the executing method.

Additional Tricky Practice Answers

Answer A: Exclude `clearCache()`

```
within(com. .student.service..*)  
&& !execution(* clearCache())
```

One-line form:

```
within(com. .student.service..*) && !execution(* clea  
rCache())
```

This matches all service-layer methods except a zero-param

```
clearCache.
```

To exclude every overloaded method named `clearCache`, regular use:

```
within(com. .... .student.service..*)  
&& !execution(* clearCache(..))
```

Answer B: Match Controllers or Services

```
within(com. .... .student.controller..*)  
||  
within(com. .... .student.service..*)
```

One-line form:

```
within(com. .... .student.controller..*) || within(com. ....  
erarmy.student.service..*)
```

A method matches when it belongs to either the controller layer or the service layer.

Answer C: Match Public Methods Except Repositories

```
within(com. .... .student..*)  
&& execution(public * *(..))  
&& !within(com. .... .student.repository..*)
```

One-line form:

```
within(com. .... .student..*) && execution(public * *(..))  
&& !within(com. .... .student.repository..*)
```

The expression requires that:

1. The method belongs to the application.
2. The method is public.
3. The method does not belong to the repository package.

Answer D: Match the `studentService` B

```
bean(studentService)
```

This matches methods executed on the Spring bean named `studentService`.

Answer E: Match Beans Ending with `Service`

```
bean(*Service)
```

This matches bean names such as:

```
studentService  
emailService  
paymentService  
notificationService
```

Bean names are case-sensitive.

Answer F: Match a Runtime `Student` Argument

```
args(com. .student.model.Student)
```

This matches a method call containing exactly one runtime argument compatible with `Student`.

For a method whose first parameter is `Student` and which may contain additional parameters, use:

```
args(com. .student.model.Student, ..)
```

Answer G: Match an Argument Annotate

@SensitiveData

```
@args(com. .student.annotation.SensitiveData)
```

This checks whether the runtime class of the passed argument is annotated with `@SensitiveData`.

It does not check an annotation written directly on the method parameter.

Answer H: Match the Actual Target Bean Type

```
target(com. .student.service.StudentService)
```

`target()` checks the type of the actual Spring bean behind the proxy.

Answer I: Match the Spring Proxy Interface

```
this(com. .student.service.StudentOperations)
```

`this()` checks whether the Spring proxy is compatible with `StudentOperations`.

With a JDK dynamic proxy, the generated proxy implements the service interface.

Answer J: Tricky Parameter Matching

Answer J1: Exactly Zero Parameters

```
execution(* *())
```

Answer J2: Exactly One Parameter of An

```
execution(* *(*))
```

Answer J3: Zero or More Parameters

```
execution(* *(..))
```

Answer J4: Exactly Two Parameters of Any Type

```
execution(* *(*, *))
```

The two `*` patterns represent exactly two parameters.

Answer J5: First Parameter Is **Long**

```
execution(* *(java.lang.Long, ..))
```

This matches:

```
getStudent(Long id)
updateStudent(Long id, Student student)
someMethod(Long id, String name, int age)
```

Answer J6: Last Parameter Is **Student**

```
execution(
    * *(.., com. .... .student.model.Student)
)
```

One-line form:

```
execution(* *(.., com. .student.model.
```

Here:

```
..      → zero or more parameters before the final parameter  
Student → the last parameter must be Student
```

This matches:

```
createStudent(Student student)
```

```
updateStudent(Long id, Student student)
```

```
saveStudent(  
    String createdBy,  
    Long departmentId,  
    Student student  
)
```

Challenge Question Answers

Challenge 1: Public Service Methods Except Delete Operations

```
within(com. .student.service..*)  
&& execution(public * *(..))  
&& !execution(* delete*(..))
```

One-line form:

```
within(com.____.student.service..*) && ex  
* *(..)) && !execution(* delete*(..))
```

This excludes method names such as:

```
deleteStudent()  
deleteAllStudents()  
deletePayment()
```

Challenge 2: Getter Methods with No Parameters

```
execution(* get*(..))
```

```
*      → any return type  
get*   → method name beginning with get  
( )    → exactly zero parameters
```

Challenge 3: Methods Returning **Student**

```
execution(  
    com.____.student.model.Student *(..)  
)
```

One-line form:

```
execution(com.____.student.model.Student *(..))
```

This matches any method returning exactly **Student**, regardless of its name, class or parameters.

It does not match:

```
List<Student>
Optional<Student>
void
boolean
```

Challenge 4: Methods with **Long** and **Student** Parameters

```
execution(
    *(java.lang.Long, com. student.model.Student)
)
```

One-line form:

```
execution(*(java.lang.Long, com. student.model.Student))
```

The method must contain exactly two parameters in this order:

1. Long
2. Student

Challenge 5: Audited Public Service Methods

```
within(com. student.service.*)
&& execution(public *(..))
&& @within(com. student.annotation.Audited)
```

One-line form:

```
within(com. student.service.*) && execution(public
```

```
* *(..) && @within(com. .student.anno
```

A method matches only when:

1. It belongs to the service layer.
2. It is public.
3. Its declaring class is annotated with `@Audited`.

Final Wildcard Reference

Pattern	Meaning
<code>*</code> as return type	Any return type
<code>*</code> as method name	Any method name
<code>get*</code>	Method name beginning with <code>get</code>
<code>*Student</code>	Method name ending with <code>Student</code>
<code>*Student*</code>	Method name containing <code>Student</code>
<code>()</code>	Exactly zero parameters
<code>(*)</code>	Exactly one parameter of any type
<code>(*, *)</code>	Exactly two parameters of any type
<code>(..)</code>	Zero or more parameters
<code>(Long)</code>	Exactly one <code>Long</code> parameter
<code>(Long, ..)</code>	First parameter is <code>Long</code>
<code>(.., Student)</code>	Last parameter is <code>Student</code>
<code>service.*</code>	Types directly inside the package
<code>service..*</code>	Types inside the package and its subpackages

Important Reminder

A correct pointcut expression tells Spring:

| Which method executions should receive the advice?

The four most commonly used designators are:

```
execution()  
within()  
@annotation()  
bean()
```

Advanced designators such as `@within()`, `@args()`, `args()`, `this()`, `@target` and `target()` are mainly useful when annotation placement, runtime arguments or proxy types matter.